

Data Handling: Import, Cleaning and Visualisation

Exercise 5: Applied Data Analysis with R

Prof. Dr. Ulrich Matter

09/12/2021

Exercises¹

Exercise A

Load `tidyverse` and read `stocks.csv` into RStudio (store it in variable `stocks`). The dataset is currently in long format. Use `pivot_wider()` to transform it into wide format (separate columns for different years). Store the resulting `tibble/data.frame` in a variable called `stocks2`. Then, run the following code.

```
pivot_longer(stocks2, `2015`:`2016`, names_to = "year", values_to = "return") == stocks
```

Why is the result not identical to the original dataset (`stocks`)? In other words, why are `pivot_longer()` and `pivot_wider()` not perfectly symmetrical? (Hint: look at the variable types and think about column names.)

```
# steps as instructed
library(tidyverse)
stocks <- read_csv("../data/stocks.csv")
stocks2 <- pivot_wider(stocks, names_from = year, values_from = return)

stocks_wide_long <- pivot_longer(stocks2, `2015`:`2016`, names_to = "year",
                                values_to = "return")

# test if symmetrical
stocks_wide_long == stocks
```

```
##      half year return
## [1,] FALSE FALSE  TRUE
## [2,] FALSE FALSE FALSE
## [3,] FALSE FALSE FALSE
## [4,] FALSE FALSE  TRUE

# another way to test whether the datasets are exactly equal
identical(stocks_wide_long, stocks)
```

```
## [1] FALSE
```

In order to evaluate the differences between the two datasets `stocks` and `stocks_wide_long`, take into account the variable types and the ordering of the columns.

```
# investigate the variable types
str(stocks_wide_long)
str(stocks)
```

¹Adopted from <https://r4ds.had.co.nz/tidy-data.html>.

The variables are not of the same types. They are also given in different order.

Exercise B

Load `tidyverse` and read `countries.csv` into R (store it in variable `countries`). Then, run the following code.

```
pivot_longer(countries, 1999, 2000, names_to = "year", values_to = "cases")
```

Why does this code fail?

```
# read the data
countries <- read_csv("../data/countries.csv")

# try the example code
pivot_longer(countries, c(1999, 2000), names_to = "year", values_to = "cases")
```

The code fails because numeric values cannot be used as variable names.

```
# fix the problem
pivot_longer(countries, c(`1999`, `2000`), names_to = "year", values_to = "cases")
```

Exercise C

Load `tidyverse` and read `people.csv` into R (store it in variable `people`). Try to spread `people`. Why does the spreading fail? How could you add a new column to fix the problem?

```
# read the data
people <- read_csv("../data/people.csv")
pivot_wider(people, names_from = key, values_from = value)

# look at the dataset
view(people)
str(people)
```

The issue is that the key-variable – the identifier – contains duplicate values.

```
# fix problem:
# assume there are two Phillip Woods and mark them with a different identifier
people$id <- c(1,1,2,3,3)
pivot_wider(people, names_from = key, values_from = value)
```

Exercise D

Read `preg.csv` into RStudio (store it in variable `preg`). Tidy `preg`. Do you need to use `pivot_wider` or `pivot_longer`? What are the variables?

```
# read the data
preg <- read_csv("../data/preg.csv")

# look at the dataset
view(preg)

# use common sense ('heuristics') to figure out what information are contained
pivot_longer(preg, c("male", "female"), names_to = "gender", values_to = "count")
```

Additional Exercises (advanced)

Exercise F

You want to investigate the relationship between GDP per capita and CO2 emissions per capita across countries. On Canvas, download the dataset `co2gdp.csv`. This dataset has been downloaded from the website “Our World in Data” (<https://ourworldindata.org/grapher/co2-emissions-vs-gdp>). Investigate the relationship between the dependent variable `CO2_percapita` and the independent variable `GDP_percapita` using a linear regression of the following form:

$$\text{CO2_percapita}_i = \alpha + \beta \text{GDP_percapita}_i + u_i.$$

Estimate this linear regression by creating your own function `myols` that takes the following three arguments: (x) for the dependent variable, (y) for the independent variable, and `matrix`, which takes `TRUE` if the function estimates the coefficients using matrix algebra, and `FALSE` if the function estimates the coefficients using the sum operators. Compare the coefficients with the R built-in function `lm()`. Finally, comment and interpret the coefficients.

Hint 1 (linear regression with sum operator): the slope parameter of the linear regression is expressed as the following formula:

$$\hat{\beta} = \frac{\sum x_i y_i - N \bar{y} \bar{x}}{\sum x_i^2 - N \bar{x}^2},$$

and the constant is given by this formula:

$$\hat{\alpha} = \bar{y} - \hat{\beta} \bar{x},$$

where $\bar{x}$ gives the mean of x and $\bar{y}$ gives the mean of y .

Hint 2 (linear regression with matrix algebra): the formula for the linear regression in matrix algebra is

$$\hat{\beta} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}\mathbf{y},$$

where you find the inverse of a matrix with the command `'solve()'`, the cross-product of two matrices with `crossprod(X,Y) = t(X)%*%Y`. Also, note that you need a column of 1s in your matrix of covariates to compute the intercept.

(**Note:** this exercise is about programming and implementing the formula of the linear regression. You don't need to understand the mechanics of linear regression for the exam. This will be covered next semester in the course **Data Analytics II**.)

```
# Create a function for linear regression
myols <- function(y, x, matrix = F) { # The function is called "myols". It takes three arguments: y, x,
  # if matrix is FALSE, we apply the formula for the slope coefficient

  # Apply formula for the slope coefficient
  N <- length(y)

  beta <- (sum(y*x) - N*mean(x)*mean(y)) / (sum(x^2)-N*mean(x)^2) # apply the formula given in the exer

  # Apply formula for the constant
  alpha <- mean(y)-beta*mean(x)

  return(list("alpha" = alpha, "beta" = beta)) # The function returns a list with two named elements
}

if (isTRUE(matrix)){ # if matrix is TRUE, we apply the formula for the slope coefficient in matrix no
```

```

    # Compute cross products and inverse
    x <- cbind(1, x) # we first need to add a column of 1 for the constant
    XXi <- solve(crossprod(x,x)) # we apply the formula given in the exercise
    Xy <- crossprod(x, y) # idem

    return(XXi %*% Xy) # idem
  }
} # The function is closed.

# Import data
co2 <- read_csv("../data/co2gdp.csv")

# Run the function
myols(y = co2$CO2_percapita, x = co2$GDP_percapita, matrix = F)

## $alpha
## [1] 0.4931492
##
## $beta
## [1] 0.000379723

myols(y = co2$CO2_percapita, x = co2$GDP_percapita, matrix = T)

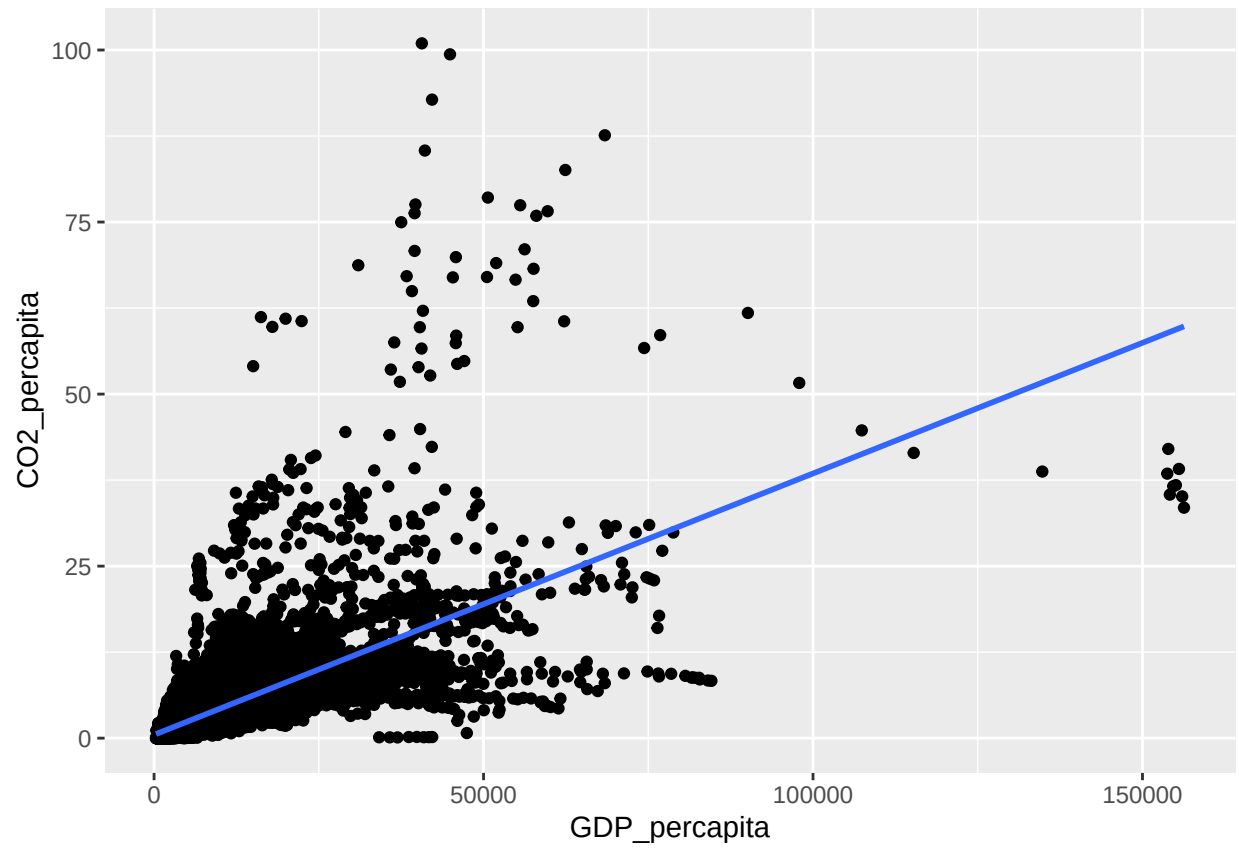
##           [,1]
## 0.493149193
## x 0.000379723

# Check with lm()
lm(CO2_percapita ~ GDP_percapita, data = co2)

##
## Call:
## lm(formula = CO2_percapita ~ GDP_percapita, data = co2)
##
## Coefficients:
## (Intercept) GDP_percapita
## 0.4931492 0.0003797

# Graph
ggplot(co2, aes(x = GDP_percapita, y = CO2_percapita)) +
  geom_point() +
  geom_smooth(method = "lm", se = F)

```



References